

```
!pip install yfinance stable-baselines3 gym matplotlib pandas numpy scikit-learn
```

```
Requirement already satisfied: yfinance in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: stable-baselines3 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: gym in /usr/local/lib/python3.12/dist-packages (0.25.
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: requests>=2.31 in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: multitasking>=0.0.7 in /usr/local/lib/python3.12/dist
Requirement already satisfied: platformdirs>=2.0.0 in /usr/local/lib/python3.12/dist
Requirement already satisfied: pytz>=2022.5 in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: frozendict>=2.3.4 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: peewee>=3.16.2 in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: beautifulsoup4>=4.11.1 in /usr/local/lib/python3.12/d
Requirement already satisfied: curl_cffi>=0.7 in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: protobuf>=3.19.0 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: websockets>=13.0 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: gymnasium<1.3.0,>=0.29.1 in /usr/local/lib/python3.12
Requirement already satisfied: torch<3.0,>=2.3 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: cloudpickle in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: gym-notices>=0.0.4 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: cycycler>=0.10 in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dis
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dis
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: typing-extensions>=4.0.0 in /usr/local/lib/python3.12
Requirement already satisfied: cffi>=2.0.0 in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: certifi>=2024.2.2 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: farama-notifications>=0.0.1 in /usr/local/lib/python3
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: pycparser in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/di
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages
```

```
import yfinance as yf
import pandas as pd
```

```
import numpy as np
```

```
data = yf.download("AAPL", start="2020-01-01", end="2023-01-01")
data = data[['Open', 'High', 'Low', 'Close', 'Volume']]
data.dropna(inplace=True)

data.head()
```

```
/tmp/ipykernel_5029/1249609047.py:1: FutureWarning: YF.download() has changed argument
data = yf.download("AAPL", start="2020-01-01", end="2023-01-01")
[*****100%*****] 1 of 1 completed/usr/local/lib/p
return datetime.utcnow().replace(tzinfo=utc)
```

```
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWa
return datetime.utcnow().replace(tzinfo=utc)
```

Price	Open	High	Low	Close	Volume	
Ticker	AAPL	AAPL	AAPL	AAPL	AAPL	
Date						

2020-01-02	71.409778	72.460776	71.156674	72.400513	135480400
2020-01-03	71.629160	72.455974	71.472477	71.696655	146322800
2020-01-06	70.819216	72.306514	70.568518	72.267944	118387200
2020-01-07	72.277548	72.533064	71.708665	71.928024	108872000
2020-01-08	71.631544	73.386416	71.631544	73.085098	132079200

```
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWa
return datetime.utcnow().replace(tzinfo=utc)
```

Next steps:

[Generate code with data](#)
[New interactive sheet](#)

```
data['returns'] = data['Close'].pct_change()
data['ma_10'] = data['Close'].rolling(10).mean()
data['ma_20'] = data['Close'].rolling(20).mean()

data.dropna(inplace=True)
```

```
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWa
return datetime.utcnow().replace(tzinfo=utc)
```

Used returns and moving averages to capture price trends.

```
from sklearn.cluster import KMeans

features = data[['returns', 'ma_10', 'ma_20']]

kmeans = KMeans(n_clusters=5)
data['pattern'] = kmeans.fit_predict(features)
```

```
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWa
return datetime.utcnow().replace(tzinfo=utc)
```

```

import gym
from gym import spaces

class TradingEnv(gym.Env):
    def __init__(self, data):
        super().__init__()
        self.data = data.reset_index()
        self.current_step = 0
        self.action_space = spaces.Discrete(4)
        self.observation_space = spaces.Box(low=-np.inf, high=np.inf, shape=(4,))
        self.position = 0
        self.entry_price = 0

    def reset(self):
        self.current_step = 0
        self.position = 0
        return self._get_obs()

    def _get_obs(self):
        row = self.data.iloc[self.current_step]
        # Explicitly convert to Python floats to ensure scalar values for the observer
        returns = float(row['returns'])
        ma_10 = float(row['ma_10'])
        ma_20 = float(row['ma_20'])
        pattern = float(row['pattern'])

        return np.array([returns, ma_10, ma_20, pattern], dtype=np.float32)

    def step(self, action):
        reward = 0
        done = False
        price = float(self.data.iloc[self.current_step]['Close'])

        if action == 1 and self.position == 0:
            self.position = 1
            self.entry_price = price
        elif action == 2 and self.position == 0:
            self.position = -1
            self.entry_price = price
        elif action == 3 and self.position != 0:
            pnl = (price - self.entry_price) * self.position
            reward = pnl / (self.entry_price * 0.01)
            reward -= 0.01
            self.position = 0

        self.current_step += 1
        if self.current_step >= len(self.data)-1:
            done = True

        return self._get_obs(), reward, done, {}

```

```

import shimmy
from stable_baselines3 import PPO

env = TradingEnv(data)

```

```

model = PPO("MlpPolicy", env, verbose=1)
model.learn(total_timesteps=10000)

model.save("trading_model")

```

iterations	3
time_elapsed	24
total_timesteps	6144
train/	
approx_kl	0.0044684475
clip_fraction	0.0238
clip_range	0.2
entropy_loss	-1.37
explained_variance	-0.00821
learning_rate	0.0003
loss	8.18
n_updates	20
policy_gradient_loss	-0.000831
value_loss	23.3

rollout/	
ep_len_mean	736
ep_rew_mean	6.8
time/	
fps	268
iterations	4
time_elapsed	30
total_timesteps	8192
train/	
approx_kl	0.008724852
clip_fraction	0.0188
clip_range	0.2
entropy_loss	-1.37
explained_variance	0.033
learning_rate	0.0003
loss	13.3
n_updates	30
policy_gradient_loss	-0.00116
value_loss	20.9

rollout/	
ep_len_mean	736
ep_rew_mean	8.95
time/	
fps	275
iterations	5
time_elapsed	37
total_timesteps	10240
train/	
approx_kl	0.009019226
clip_fraction	0.0646
clip_range	0.2
entropy_loss	-1.35
explained_variance	0.0466
learning_rate	0.0003
loss	15.2
n_updates	40
policy_gradient_loss	-0.00476
value_loss	28

✓ Test the Trained Agent

Now, let's evaluate the performance of our trained PPO agent.

```
from stable_baselines3 import PPO
import matplotlib.pyplot as plt

# Load the trained model
model = PPO.load("trading_model")

# Create a fresh environment for evaluation
env = TradingEnv(data)

obs = env.reset()
total_reward = 0
history = []

for i in range(len(env.data) - 1):
    action, _states = model.predict(obs, deterministic=True)
    obs, reward, done, info = env.step(action)
    total_reward += reward
    history.append({'step': i, 'action': action, 'reward': reward, 'total_reward':
    if done:
        break

print(f"Total reward during testing: {total_reward}")

history_df = pd.DataFrame(history)

plt.figure(figsize=(15, 7))

# Plot Total Reward
plt.subplot(2, 1, 1)
plt.plot(history_df['step'], history_df['total_reward'], label='Cumulative Reward')
plt.title('Cumulative Reward Over Time')
plt.xlabel('Time Step')
plt.ylabel('Reward')
plt.legend()
plt.grid(True)

# Plot Actions and Close Price
plt.subplot(2, 1, 2)
plt.plot(history_df['step'], history_df['close_price'], label='Close Price', color=

buy_signals = history_df[history_df['action'] == 1]
sell_signals = history_df[history_df['action'] == 2]
close_trade_signals = history_df[history_df['action'] == 3]

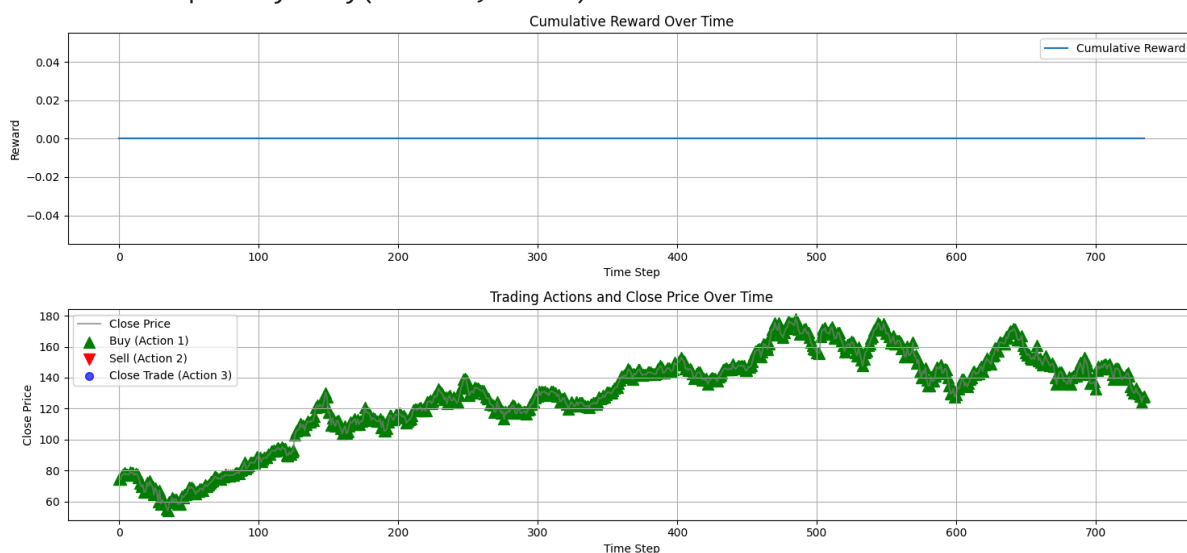
plt.scatter(buy_signals['step'], buy_signals['close_price'], marker='^', color='gre
plt.scatter(sell_signals['step'], sell_signals['close_price'], marker='v', color='r
plt.scatter(close_trade_signals['step'], close_trade_signals['close_price'], marker

plt.title('Trading Actions and Close Price Over Time')
plt.xlabel('Time Step')
plt.ylabel('Close Price')
plt.legend()
```

```
plt.grid(True)
```

```
plt.tight_layout()
plt.show()
```

```
/tmp/ipykernel_5029/4167578411.py:22: FutureWarning: Calling float on a single element
returns = float(row['returns'])
/tmp/ipykernel_5029/4167578411.py:23: FutureWarning: Calling float on a single element
ma_10 = float(row['ma_10'])
/tmp/ipykernel_5029/4167578411.py:24: FutureWarning: Calling float on a single element
ma_20 = float(row['ma_20'])
/tmp/ipykernel_5029/4167578411.py:25: FutureWarning: Calling float on a single element
pattern = float(row['pattern'])
/tmp/ipykernel_5029/4167578411.py:32: FutureWarning: Calling float on a single element
price = float(self.data.iloc[self.current_step]['Close'])
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning:
return datetime.utcnow().replace(tzinfo=utc)
Total reward during testing: 0
/usr/local/lib/python3.12/dist-packages/matplotlib/cbook.py:1709: FutureWarning: Call
return math.isfinite(val)
/usr/local/lib/python3.12/dist-packages/matplotlib/cbook.py:1345: FutureWarning: Call
return np.asarray(x, float)
/usr/local/lib/python3.12/dist-packages/matplotlib/collections.py:200: FutureWarning
offsets = np.asarray(offsets, float)
```



```
!pip install yfinance stable-baselines3 gym matplotlib pandas numpy scikit-learn shi
```

